SEMANA 7

Miguel Andres Gomez

(Estudiante)

Jesus Ariel Gonzalez Bonilla

(profesor)

25/03/2026

Corhuila

Contenido

Introducción 3

Crear un trigger **BEFORE** para normalizar el nombre del estudiante (TRIM + INITCAP). 4

Crear un trigger **BEFORE** para asignar updated_at automáticamente en UPDATE..... 4

Crear un trigger **BEFORE** que valide nota_final entre 0 y 5. 5

Crear un trigger **AFTER** que registre auditoría en una tabla audit (INSERT/UPDATE/DELETE). .. 6

Ejecutar mínimo 4 pruebas: 1 insert ok, 1 update ok, 1 delete (opcional), 1 operación fallida por validación. 7

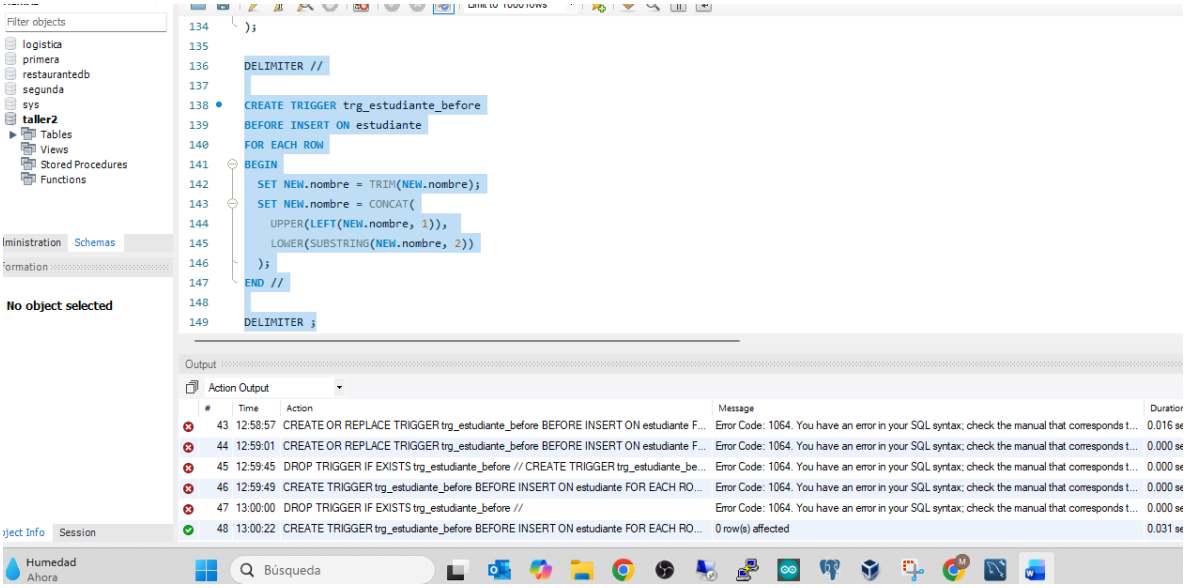
Conclusión 8

Introducción

En esta práctica se implementaron triggers en MySQL para fortalecer el control y la integridad de los datos dentro de una base de datos. Se desarrolló un trigger BEFORE para validar que la nota_final estuviera dentro del rango permitido (0 a 5), evitando el ingreso de datos incorrectos. Además, se crearon triggers AFTER para registrar auditorías de las operaciones realizadas (INSERT, UPDATE y DELETE), permitiendo llevar un control detallado de los cambios en la tabla matrícula.

Crear un trigger **BEFORE** para normalizar el nombre del estudiante (TRIM + INITCAP).

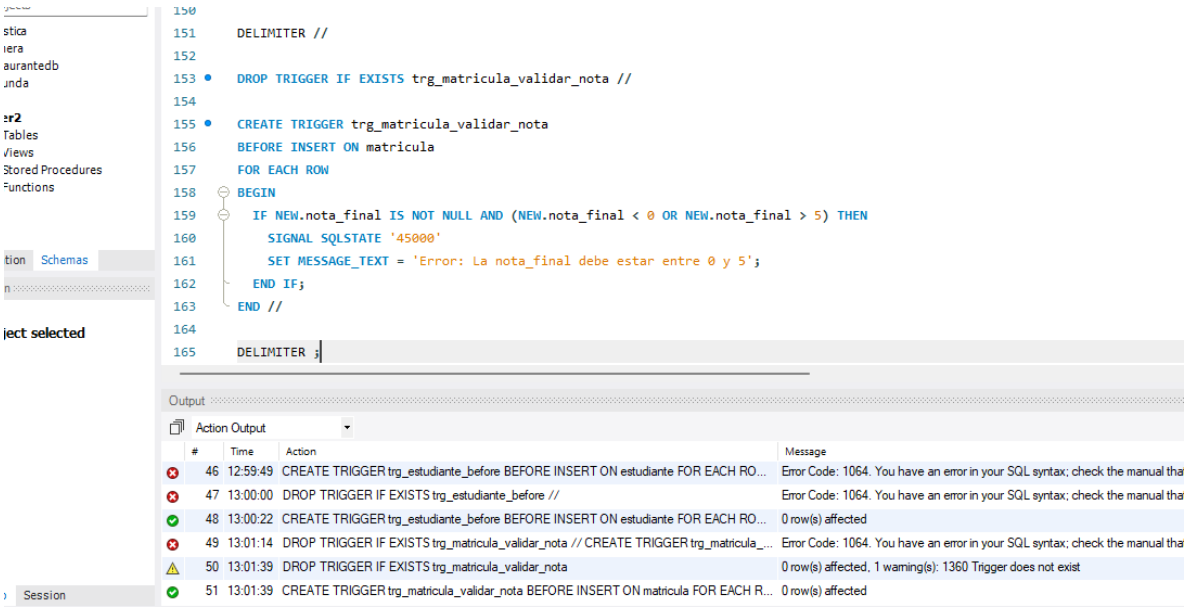
```
DELIMITER //

CREATE TRIGGER trg_estudiante_before
BEFORE INSERT ON estudiante
FOR EACH ROW
BEGIN
SET NEW.nombre = TRIM(NEW.nombre);
SET NEW.nombre = CONCAT(
UPPER(LEFT(NEW.nombre, 1)),
LOWER(SUBSTRING(NEW.nombre, 2))
);
END //

DELIMITER ;
```



Crear un trigger **BEFORE** para asignar updated_at automáticamente en UPDATE.

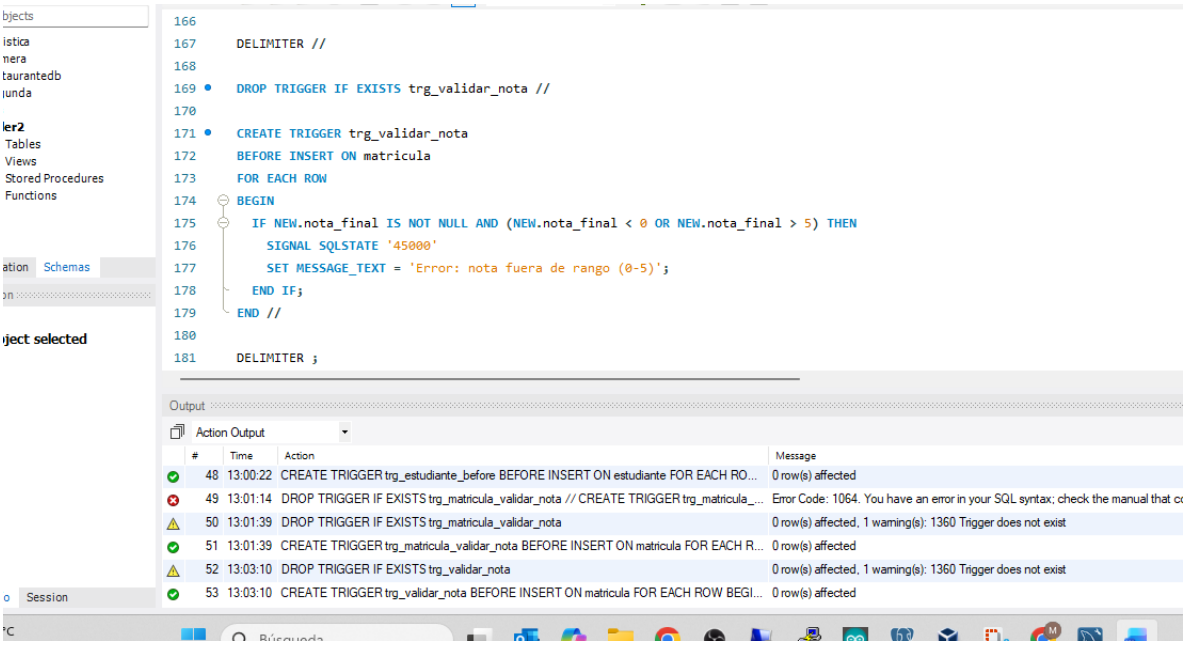
```
DELIMITER //

DROP TRIGGER IF EXISTS trg_matricula_validar_nota //

CREATE TRIGGER trg_matricula_validar_nota
BEFORE INSERT ON matricula
FOR EACH ROW
BEGIN
  IF NEW.nota_final IS NOT NULL AND (NEW.nota_final < 0 OR NEW.nota_final > 5) THEN
```

```
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Error: La nota_final debe estar entre 0 y 5';
 END IF;
END //

DELIMITER ;
```

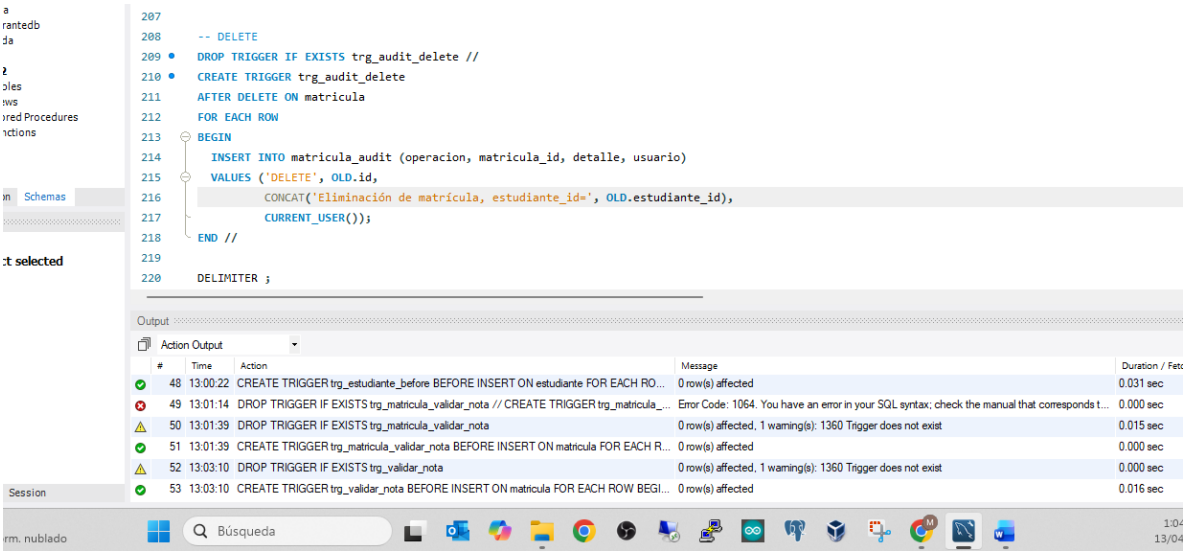


Crear un trigger **BEFORE** que valide nota_final entre 0 y 5.

```
DELIMITER //

DROP TRIGGER IF EXISTS trg_validar_nota //

CREATE TRIGGER trg_validar_nota
BEFORE INSERT ON matricula
FOR EACH ROW
BEGIN
 IF NEW.nota_final IS NOT NULL AND (NEW.nota_final < 0 OR NEW.nota_final > 5) THEN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Error: nota fuera de rango (0-5)';
 END IF;
END //

DELIMITER ;
```



Crear un trigger **AFTER** que registre auditoría en una tabla audit (INSERT/UPDATE/DELETE).



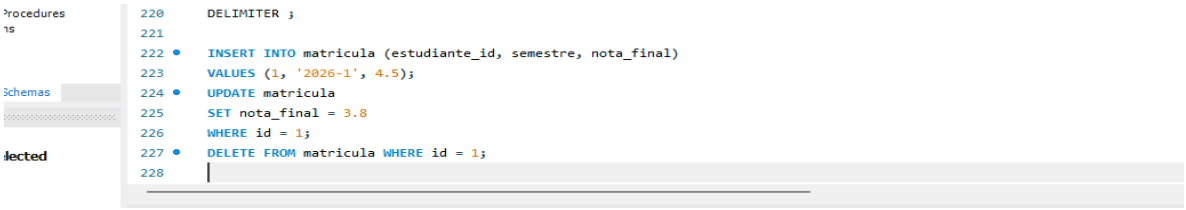
DELIMITER //

-- INSERT
DROP TRIGGER IF EXISTS trg_audit_insert //
CREATE TRIGGER trg_audit_insert
AFTER INSERT ON matricula
FOR EACH ROW
BEGIN
 INSERT INTO matricula_audit (operacion, matricula_id, detalle, usuario)

```
 VALUES ('INSERT', NEW.id,
       CONCAT('Insert en matrícula, estudiante_id=', NEW.estudiante_id),
       CURRENT_USER());
END //

-- UPDATE
DROP TRIGGER IF EXISTS trg_audit_update //
CREATE TRIGGER trg_audit_update
AFTER UPDATE ON matricula
FOR EACH ROW
BEGIN
 INSERT INTO matricula_audit (operacion, matricula_id, detalle, usuario)
 VALUES ('UPDATE', NEW.id,
       CONCAT('Nota actualizada a ', NEW.nota_final),
       CURRENT_USER());
END //

-- DELETE
DROP TRIGGER IF EXISTS trg_audit_delete //
CREATE TRIGGER trg_audit_delete
AFTER DELETE ON matricula
FOR EACH ROW
BEGIN
 INSERT INTO matricula_audit (operacion, matricula_id, detalle, usuario)
 VALUES ('DELETE', OLD.id,
       CONCAT('Eliminación de matrícula, estudiante_id=', OLD.estudiante_id),
       CURRENT_USER());
END //

DELIMITER ;
```

Ejecutar mínimo 4 pruebas: 1 insert ok, 1 update ok, 1 delete (opcional), 1 operación fallida por validación.

```
INSERT INTO matricula (estudiante_id, semestre, nota_final)
VALUES (1, '2026-1', 4.5);
UPDATE matricula
SET nota_final = 3.8
WHERE id = 1;
DELETE FROM matricula WHERE id = 1;
```



Conclusión

El uso de triggers permitió automatizar validaciones y garantizar la consistencia de la información sin depender únicamente de la aplicación. Asimismo, la implementación de auditorías demostró ser una buena práctica para el seguimiento de cambios y la seguridad de los datos. Esta práctica ayudó a comprender la importancia de los mecanismos automáticos en bases de datos para prevenir errores y mejorar la confiabilidad del sistema.